

CS254 Database Systems — Project

Fusion Bites (ProjectDB)

Faizan Muhammad BSCS25040

Abdul Moiz BSCS25133

Talha Sami BSCS116

Shahbaz Hussain BSCS25130

May 10, 2026

Contents

Academic Integrity Declaration	3
AI Disclosure	3
1 Task 1a — Introduction & Initial Analysis (Non-Technical)	4
1.1 Organisation Overview	4
1.2 What the Database Will Do	4
1.3 Daily Business Transactions Supported	4
1.4 Entities Included (Minimum 8)	4
1.5 Relational Database: Advantages & Disadvantages	4
2 Task 1b — ERD & Normalisation to 3NF	5
2.1 Entity Relationship Diagram	5
2.2 Normalisation (UNF $\rightarrow$ 1NF $\rightarrow$ 2NF $\rightarrow$ 3NF)	7
2.3 Problems Encountered (ERD/Normalisation)	7
3 Task 1c — Data Dictionary	8
4 Task 2a — Frontend-to-Backend-to-Database Architecture	13
4.1 Three-Tier Architecture	13
4.2 Tier Roles	13
4.3 Connection Pool (src/lib/db.ts)	13
4.4 Example: Menu (app/api/menu/route.ts)	13
4.5 Example: Place Order (app/api/orders/route.ts)	14
4.6 Request Flow	14
5 Task 2b — Application Screenshots	15

6	Task 2c — Five SQL Queries	18
6.1	Query 1 — High-value active orders (CTE + JOIN)	18
6.2	Query 2 — Menu summary by category (aggregation)	19
6.3	Query 3 — Advance order status (UPDATE)	19
6.4	Query 4 — Delete expired inactive coupons (DELETE)	20
6.5	Query 5 — Customer leaderboard (aggregation + HAVING)	21

Academic Integrity Declaration

Written analysis, ERD, normalisation, SQL queries, and data dictionary are the group's own work. Any permitted tool use is disclosed below.

AI Disclosure

- **Tool(s) used:** ChatGPT, Claude, GitHub Copilot
- **Permitted usage:** Syntax help for LaTeX, debugging Next.js and SQL errors, and explaining concepts such as UNF and 1NF.
- **Verification:** Executed scripts in MySQL, checked foreign keys against `FoodDeliverySystem.sql`, compared the ERD with the live schema, and tested queries on `ProjectDB`.

1 Task 1a — Introduction & Initial Analysis (Non-Technical)

1.1 Organisation Overview

Fusion Bites is a restaurant brand. The system supports online ordering, customer accounts, payments and invoices, delivery coordination, reviews, and admin reporting.

1.2 What the Database Will Do

The database stores authoritative data for users, menu, orders, payments, invoices, transactions, reviews, and coupons so the web app can query and update state consistently.

1.3 Daily Business Transactions Supported

- Customer registers / logs in.
- Customer browses menu by category.
- Customer places an order (header + line items).
- Delivery agent may be linked to an order.
- Payment is recorded; invoice may be generated.
- Transaction log entries for payment/refund flows.
- Customer posts a review.
- Admin views analytics and manages orders, users, payments.

1.4 Entities Included (Minimum 8)

- Users, Restaurants, DeliveryAgents, MenuItems
- Orders, OrderItems, Payments, Invoices, Transactions
- Reviews, Coupons

1.5 Relational Database: Advantages & Disadvantages

Advantages

- Integrity via keys, FKs, and constraints.
- Less redundancy through normalisation.
- Flexible reporting (aggregations, joins).
- DB transactions for multi-step updates.

Disadvantages

- Schema changes need care (migrations, app updates).
- Complex joins as the model grows.

2 Task 1b — ERD & Normalisation to 3NF

2.1 Entity Relationship Diagram

Export from diagrams.net and place the image next to this `.tex` file (or set the path in `\graphicspath`).

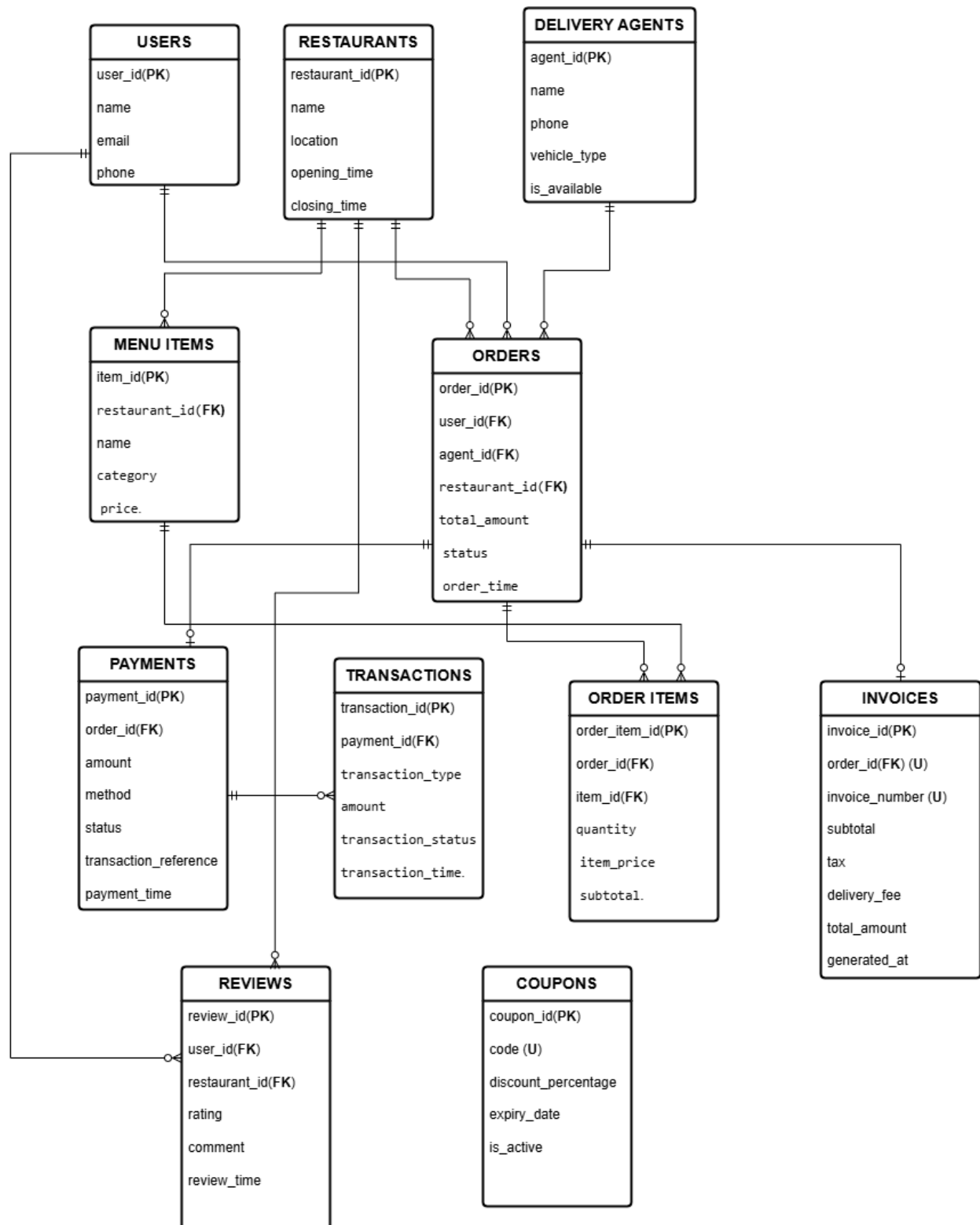


Figure 1: Fusion Bites ERD

2.2 Normalisation (UNF → 1NF → 2NF → 3NF)

Complete the cells below to match your ERD and `FoodDeliverySystem.sql`.

UNF

Item	Details

1NF

Relation	Keys / notes

2NF

Relation	Partial dependencies removed

3NF

Table	PK / FK (summary)
Users	PK: user_id
Restaurants	PK: restaurant_id
DeliveryAgents	PK: agent_id
MenuItems	PK: item_id; FK: restaurant_id
Orders	PK: order_id; FK: user_id, restaurant_id, agent_id (nullable)
OrderItems	PK: order_item_id; FK: order_id, item_id
Payments	PK: payment_id; FK: order_id (UNIQUE)
Invoices	PK: invoice_id; FK: order_id (UNIQUE)
Transactions	PK: transaction_id; FK: payment_id
Reviews	PK: review_id; FK: user_id, restaurant_id
Coupons	PK: coupon_id

2.3 Problems Encountered (ERD/Normalisation)

- Early database design stored several dishes inside a single order row, creating repeating groups and violating First Normal Form (1NF).
- Resolution: The structure was normalised by separating the data into `Orders` and `OrderItems` tables, where each ordered dish is stored as an individual row with its own quantity and price information.

3 Task 1c — Data Dictionary

Legend

Key: PK / FK. **Null:** NOT NULL / NULL per schema. **Notes:** UNIQUE, DEFAULT, CHECK, AUTO_INCREMENT, FK + ON DELETE.

Users

Attribute	Type	Key	Null	Notes
user_id	INT	PK	NOT NULL	AUTO_INCREMENT
name	VARCHAR(100)		NOT NULL	
email	VARCHAR(100)		NOT NULL	UNIQUE
password	VARCHAR(255)		NOT NULL	
role	ENUM('customer','ac		NOT NULL	DEFAULT 'customer'
phone	VARCHAR(15)		NULL	UNIQUE
address	VARCHAR(150)		NOT NULL	
created_at	TIMESTAMP		NOT NULL	DEFAULT CURRENT_TIMESTAMP

Restaurants

Attribute	Type	Key	Null	Notes
restaurant_id	INT	PK	NOT NULL	AUTO_INCREMENT
name	VARCHAR(100)		NOT NULL	
slogan	VARCHAR(255)		NULL	
location	VARCHAR(255)		NOT NULL	
phone	VARCHAR(20)		NULL	
opening_time	TIME		NULL	
closing_time	TIME		NULL	
rating	DECIMAL(2,1)		NOT NULL	DEFAULT 0.0
created_at	TIMESTAMP		NOT NULL	DEFAULT CURRENT_TIMESTAMP

DeliveryAgents

Attribute	Type	Key	Null	Notes
agent_id	INT	PK	NOT NULL	AUTO_INCREMENT
name	VARCHAR(100)		NOT NULL	
phone	VARCHAR(15)		NOT NULL	UNIQUE
vehicle_type	ENUM('Bike','Car')		NOT NULL	DEFAULT 'Bike'
is_available	BOOLEAN		NOT NULL	DEFAULT TRUE
created_at	TIMESTAMP		NOT NULL	DEFAULT CURRENT_TIMESTAMP

MenuItems

Attribute	Type	Key	Null	Notes
item_id	INT	PK	NOT NULL	AUTO_INCREMENT
restaurant_id	INT	FK	NOT NULL	REFERENCES Restaurants, ON DELETE CASCADE
name	VARCHAR(150)		NOT NULL	
category	VARCHAR(100)		NOT NULL	
description	TEXT		NULL	
price	DECIMAL(10,2)		NOT NULL	

Orders

Attribute	Type	Key	Null	Notes
order_id	INT	PK	NOT NULL	AUTO_INCREMENT
user_id	INT	FK	NOT NULL	REFERENCES Users, ON DELETE CASCADE
restaurant_id	INT	FK	NOT NULL	REFERENCES Restaurants, ON DELETE CASCADE
agent_id	INT	FK	NULL	REFERENCES DeliveryAgents, ON DELETE SET NULL

Attribute	Type	Key	Null	Notes
total_amount	DECIMAL(10,2)		NOT NULL	DEFAULT 0.00
delivery_address	VARCHAR(255)		NULL	
order_time	TIMESTAMP		NOT NULL	DEFAULT CURRENT_TIMESTAMP
status	ENUM(Placed, Preparing, ...)		NOT NULL	DEFAULT 'Placed'

OrderItems

Attribute	Type	Key	Null	Notes
order_item_id	INT	PK	NOT NULL	AUTO_INCREMENT
order_id	INT	FK	NOT NULL	REFERENCES Orders, ON DELETE CASCADE
item_id	INT	FK	NOT NULL	REFERENCES MenuItems, ON DELETE CASCADE
quantity	INT		NOT NULL	
item_price	DECIMAL(10,2)		NOT NULL	
subtotal	DECIMAL(10,2)		NOT NULL	

Payments

Attribute	Type	Key	Null	Notes
payment_id	INT	PK	NOT NULL	AUTO_INCREMENT
order_id	INT	FK	NULL	UNIQUE; REFERENCES Orders, ON DELETE CASCADE
amount	DECIMAL(10,2)		NOT NULL	
method	ENUM('Cash','Card',		NULL	
status	ENUM('Pending','Co		NOT NULL	DEFAULT 'Pending'
transaction_referenc	VARCHAR(255)		NULL	
payment_time	TIMESTAMP		NOT NULL	DEFAULT CURRENT_TIMESTAMP

Invoices

Attribute	Type	Key	Null	Notes
invoice_id	INT	PK	NOT NULL	AUTO_INCREMENT
order_id	INT	FK	NOT NULL	UNIQUE; REFERENCES Orders, ON DELETE CASCADE
invoice_number	VARCHAR(100)		NOT NULL	UNIQUE
subtotal	DECIMAL(10,2)		NOT NULL	
tax	DECIMAL(10,2)		NOT NULL	DEFAULT 0.00
delivery_fee	DECIMAL(10,2)		NOT NULL	DEFAULT 0.00
total_amount	DECIMAL(10,2)		NOT NULL	
generated_at	TIMESTAMP		NOT NULL	DEFAULT CURRENT_TIMESTAMP

Transactions

Attribute	Type	Key	Null	Notes
transaction_id	INT	PK	NOT NULL	AUTO_INCREMENT
payment_id	INT	FK	NOT NULL	REFERENCES Payments, ON DELETE CASCADE
transaction_reference	VARCHAR(255)		NOT NULL	
transaction_type	ENUM('Payment', 'Refund')		NOT NULL	DEFAULT 'Payment'
amount	DECIMAL(10,2)		NOT NULL	
transaction_status	ENUM('Pending', 'Completed')		NOT NULL	DEFAULT 'Completed'
transaction_time	TIMESTAMP		NOT NULL	DEFAULT CURRENT_TIMESTAMP

Reviews

Attribute	Type	Key	Null	Notes
review_id	INT	PK	NOT NULL	AUTO_INCREMENT
user_id	INT	FK	NOT NULL	REFERENCES Users, ON DELETE CASCADE
restaurant_id	INT	FK	NOT NULL	REFERENCES Restaurants, ON DELETE CASCADE

Attribute	Type	Key	Null	Notes
rating	INT		NOT NULL	CHECK (rating $\geq$ 1 AND rating $\leq$ 5)
comment	TEXT		NULL	
review_time	TIMESTAMP		NOT NULL	DEFAULT CURRENT_TIMESTAMP

Coupons

Attribute	Type	Key	Null	Notes
coupon_id	INT	PK	NOT NULL	AUTO_INCREMENT
code	VARCHAR(50)		NOT NULL	UNIQUE
discount_percentage	INT		NOT NULL	
expiry_date	DATE		NULL	
is_active	BOOLEAN		NOT NULL	DEFAULT TRUE

4 Task 2a — Frontend-to-Backend-to-Database Architecture

4.1 Three-Tier Architecture

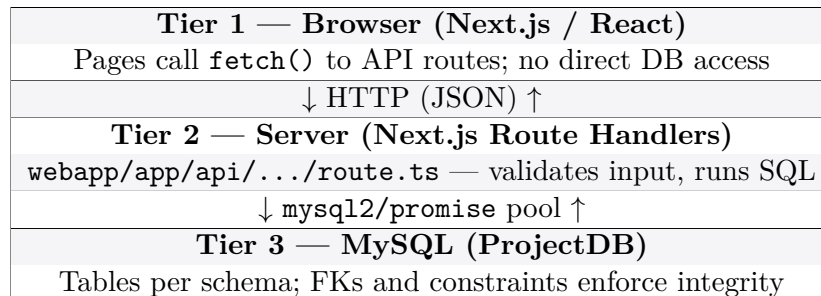


Figure 2: Three-tier architecture

4.2 Tier Roles

Browser: Renders UI; calls endpoints (e.g. `GET /api/menu`, `POST /api/orders`).

Server: Parameterised queries via shared pool (`src/lib/db.ts`); orders use a transaction (`beginTransaction / commit / rollback`).

Database: Single source of truth for users, menu, orders, payments, etc.

4.3 Connection Pool (`src/lib/db.ts`)

```
1 import mysql from "mysql2/promise";
2
3 const pool = mysql.createPool({
4   host: process.env.DB_HOST ?? "localhost",
5   user: process.env.DB_USER!,
6   password: process.env.DB_PASSWORD!,
7   database: "ProjectDB",
8   waitForConnections: true,
9   connectionLimit: 10,
10 });
11
12 export default pool;
```

Listing 1: Shared MySQL pool (credentials must not be committed; use env or secure config)

4.4 Example: Menu (`app/api/menu/route.ts`)

```
1 import { NextResponse } from "next/server";
2 import pool from "@/src/lib/db";
3
4 export async function GET() {
5   const [rows] = await pool.query(`
6     SELECT item_id, name, category, description, price
7     FROM MenuItems
```

```

8     ORDER BY category, name
9     ');
10    return NextResponse.json(rows);
11 }

```

Listing 2: GET menu — matches project structure

4.5 Example: Place Order (`app/api/orders/route.ts`)

```

1  await connection.beginTransaction();
2  // INSERT Orders (... total_amount, status, ...)
3  for (const item of items) {
4      await connection.query(
5          'INSERT INTO OrderItems (order_id, item_id, quantity, item_price,
6              subtotal)
7              VALUES (?, ?, ?, ?, ?)',
8          [orderId, item.item_id, item.quantity, item.price, subtotal]
9      );
10 }
11 await connection.query(
12     'INSERT INTO Payments (order_id, amount, method, status,
13         transaction_reference)
14         VALUES (?, ?, ?, ?, ?)',
15     [orderId, totalAmount, payment_method, "Completed", ref]
16 );
17 await connection.commit();

```

Listing 3: POST order — transactional pattern (simplified)

4.6 Request Flow

Browser → API route → parameterised SQL → JSON response → UI update.

5 Task 2b — Application Screenshots

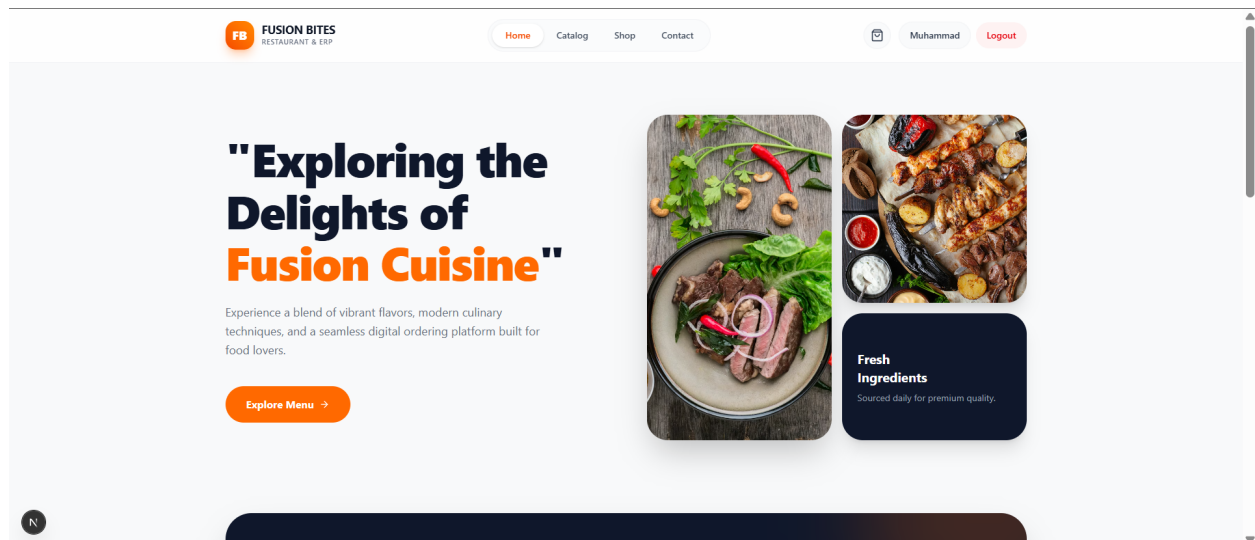


Figure 3: Homepage / menu browse

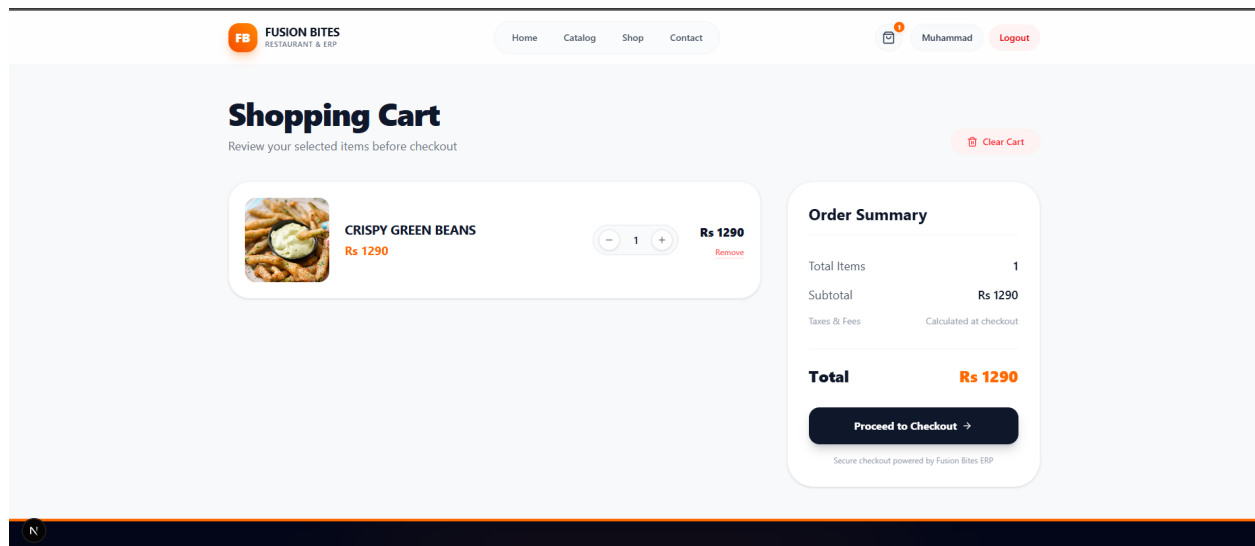


Figure 4: Order placement

Order Summary

1x

Crispy Green Beans

Rs 1290

Subtotal

Rs 1290

Delivery Fee

FREE

Total

Rs 1290

Confirm & Pay Rs 1290 →

✓ Secure checkout powered by Fusion Bites

Figure 5: Invoice

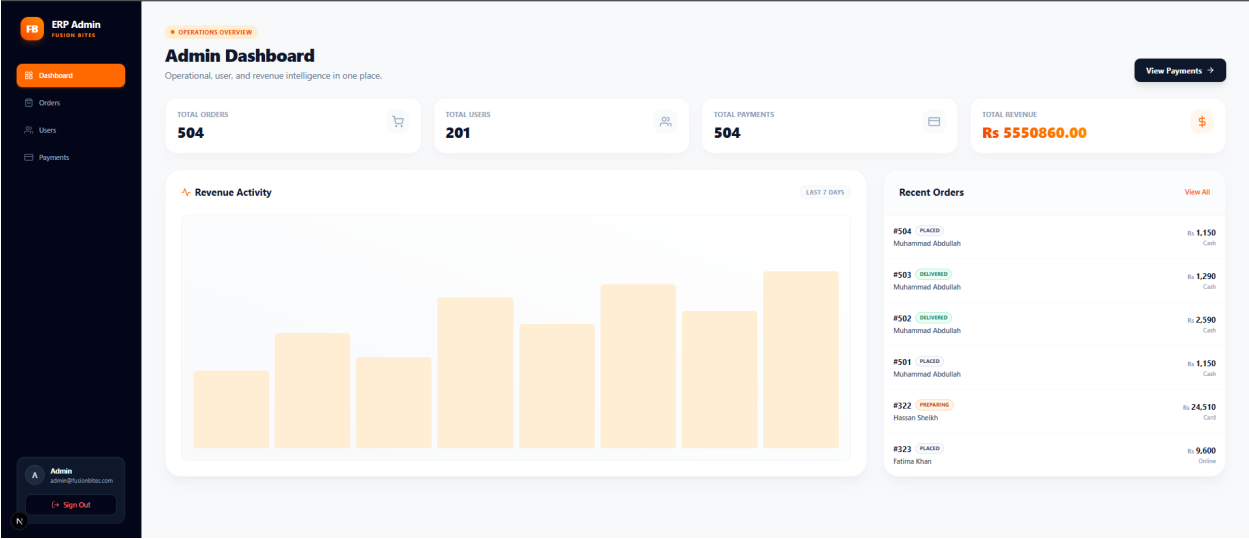


Figure 6: Admin analytics

6 Task 2c — Five SQL Queries

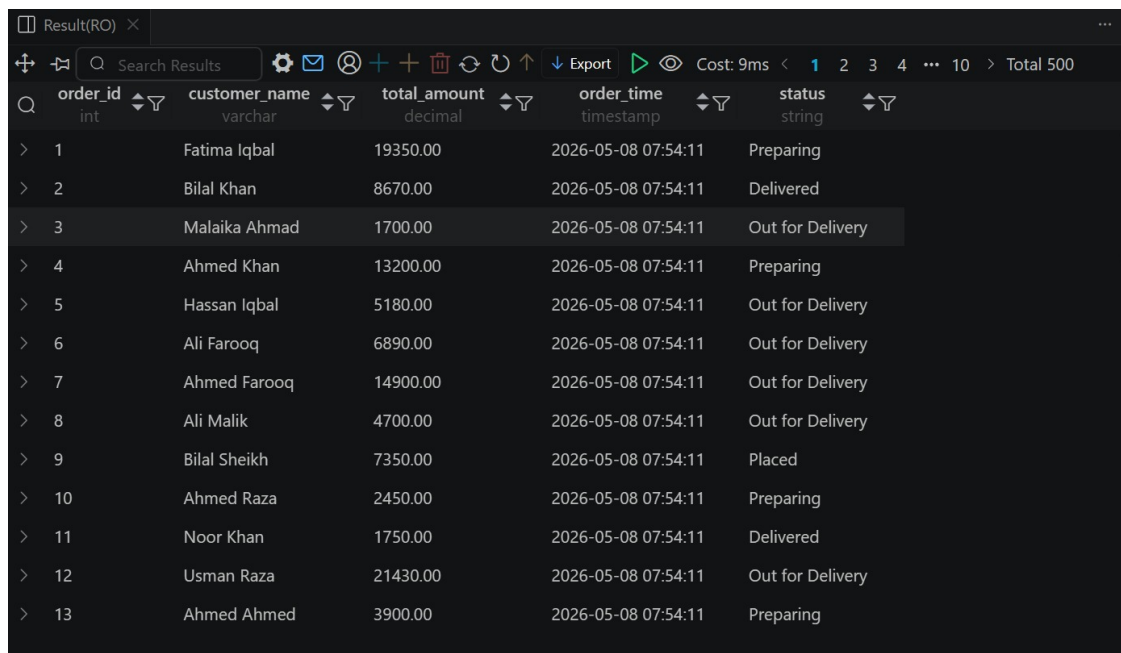
Run after `USE ProjectDB;`. Replace literal IDs with values that exist in your database.

Covers: `SELECT`, `UPDATE`, `DELETE`, `JOIN`, `WHERE`, `AND`, `COUNT`, `ORDER BY ASC`, one **CTE**, and one **aggregation** query.

6.1 Query 1 — High-value active orders (CTE + JOIN)

Non-cancelled orders over Rs. 500 with customer name; oldest first.

```
1  USE ProjectDB;
2
3  WITH priced_orders AS (
4      SELECT order_id, user_id, total_amount, order_time, status
5      FROM Orders
6      WHERE status IN ('Placed', 'Preparing', 'Out for Delivery', 'Delivered')
7  )
8  SELECT po.order_id, u.name AS customer_name, po.total_amount, po.
9      order_time, po.status
10 FROM priced_orders po
11 JOIN Users u ON u.user_id = po.user_id
12 WHERE u.role = 'customer' AND po.total_amount > 500
13 ORDER BY po.order_time ASC
14 LIMIT 50;
```



The screenshot shows a database query result in a dark-themed interface. The window title is "Result(RO)". The toolbar includes icons for search, settings, and other database functions. The query cost is 9ms, and the total number of rows is 500. The result is displayed as a table with the following columns: order_id (int), customer_name (varchar), total_amount (decimal), order_time (timestamp), and status (string). The table contains 13 rows of data, with the third row highlighted.

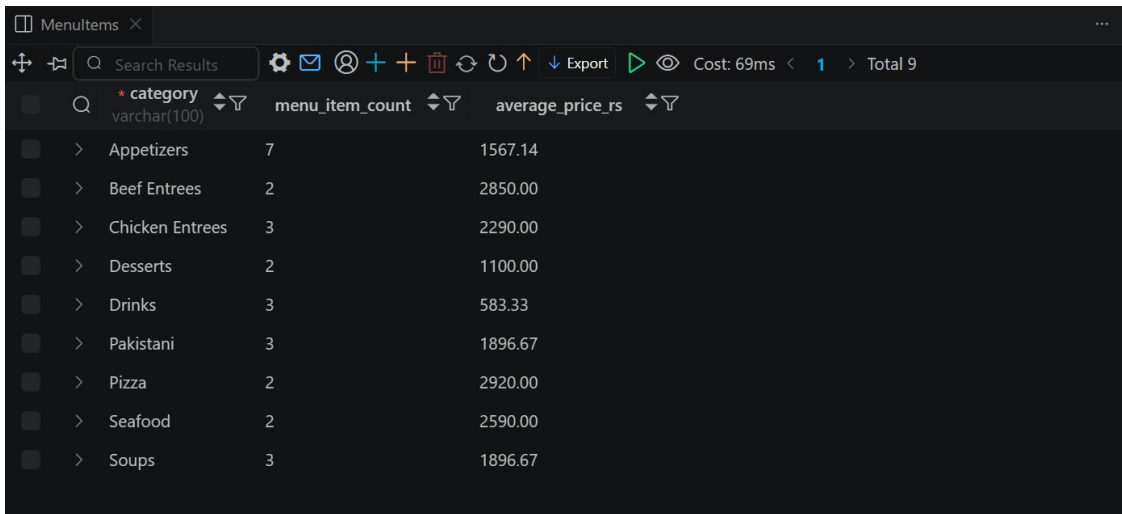
order_id	customer_name	total_amount	order_time	status
> 1	Fatima Iqbal	19350.00	2026-05-08 07:54:11	Preparing
> 2	Bilal Khan	8670.00	2026-05-08 07:54:11	Delivered
> 3	Malaika Ahmad	1700.00	2026-05-08 07:54:11	Out for Delivery
> 4	Ahmed Khan	13200.00	2026-05-08 07:54:11	Preparing
> 5	Hassan Iqbal	5180.00	2026-05-08 07:54:11	Out for Delivery
> 6	Ali Farooq	6890.00	2026-05-08 07:54:11	Out for Delivery
> 7	Ahmed Farooq	14900.00	2026-05-08 07:54:11	Out for Delivery
> 8	Ali Malik	4700.00	2026-05-08 07:54:11	Out for Delivery
> 9	Bilal Sheikh	7350.00	2026-05-08 07:54:11	Placed
> 10	Ahmed Raza	2450.00	2026-05-08 07:54:11	Preparing
> 11	Noor Khan	1750.00	2026-05-08 07:54:11	Delivered
> 12	Usman Raza	21430.00	2026-05-08 07:54:11	Out for Delivery
> 13	Ahmed Ahmed	3900.00	2026-05-08 07:54:11	Preparing

Figure 7: Query 1 — results

6.2 Query 2 — Menu summary by category (aggregation)

Count and average price per category.

```
1 SELECT category,
2     COUNT(*) AS menu_item_count,
3     ROUND(AVG(price), 2) AS average_price_rs
4 FROM MenuItems
5 WHERE price > 0 AND category IS NOT NULL
6 GROUP BY category
7 ORDER BY category ASC;
```



	* category varchar(100)	menu_item_count	average_price_rs
>	Appetizers	7	1567.14
>	Beef Entrees	2	2850.00
>	Chicken Entrees	3	2290.00
>	Desserts	2	1100.00
>	Drinks	3	583.33
>	Pakistani	3	1896.67
>	Pizza	2	2920.00
>	Seafood	2	2590.00
>	Soups	3	1896.67

Figure 8: Query 2 — results

6.3 Query 3 — Advance order status (UPDATE)

```
1 UPDATE Orders
2 SET status = 'Preparing'
3 WHERE order_id = 1 AND status = 'Placed';
```

```

database > submission > task2c_five_queries.sql > ...
58  -----
59
60  >Run | +Tab | JSON
61  SELECT status FROM Orders
62  WHERE order_id = 14; 2ms
63
64  >Run | Select
65  UPDATE Orders
66  SET status = 'Preparing'
67  WHERE order_id = 14
68      AND status = 'Placed';
69

```

Orders x Orders

Search Results

status
enum('Placed','Preparing')

Preparing

Cost: 22ms < 1 > Total 1

Figure 9: Query 3 — results

6.4 Query 4 — Delete expired inactive coupons (DELETE)

```

1 DELETE FROM Coupons
2 WHERE is_active = FALSE
3     AND expiry_date IS NOT NULL
4     AND expiry_date < CURDATE();

```

```

74
75  >Run | Select
76  DELETE FROM Coupons
77  WHERE is_active = FALSE
78      AND expiry_date IS NOT NULL
79      AND expiry_date < CURDATE(); 209ms
80
81  -----
82  -- Query 5 - SELECT + FROM + JOIN + WHERE + COUNT (window) pattern
83  -- Uses COUNT(...) OVER (...) for ranking customers by orders
84  -- -----

```

Coupons x Orders

Search Results

Cost: 209ms < >

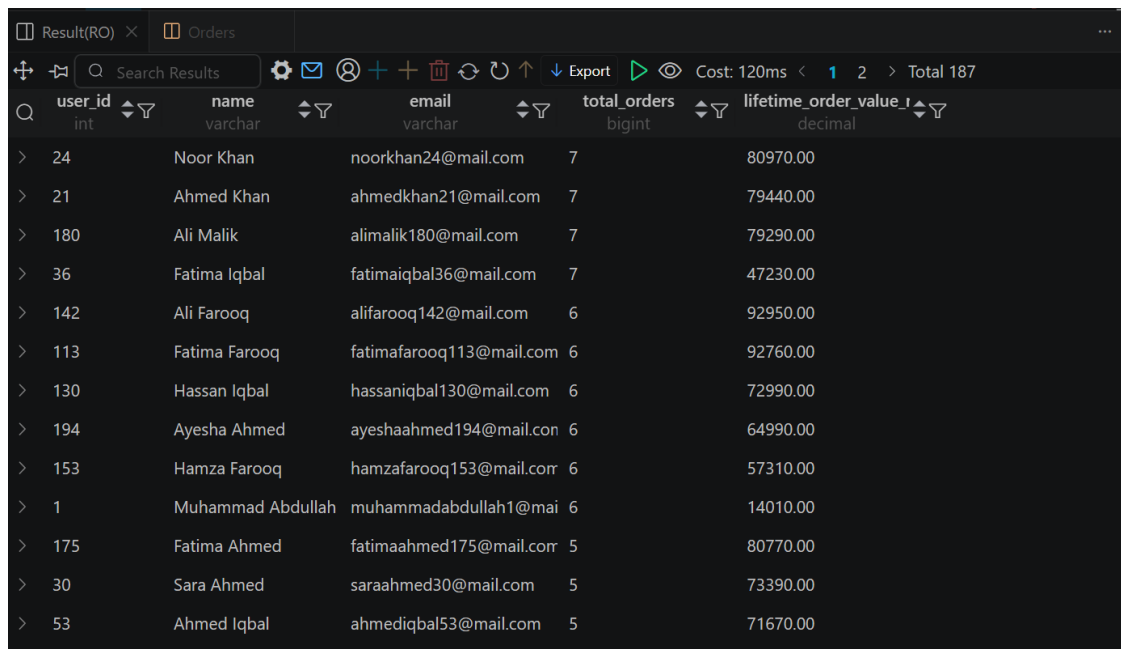
DELETE FROM Coupons WHERE is_active = FALSE AND expiry_date IS NOT NULL AND expiry_date < CURDATE()

AffectedRows : 0

Figure 10: Query 4 — results

6.5 Query 5 — Customer leaderboard (aggregation + HAVING)

```
1 SELECT u.user_id, u.name, u.email,  
2        COUNT(o.order_id) AS total_orders,  
3        ROUND(SUM(IFNULL(o.total_amount, 0)), 2) AS  
4        lifetime_order_value_rs  
5 FROM Users u  
6 LEFT JOIN Orders o ON o.user_id = u.user_id  
7 WHERE u.role = 'customer'  
8 GROUP BY u.user_id, u.name, u.email  
9 HAVING COUNT(o.order_id) > 0  
10 ORDER BY total_orders DESC, lifetime_order_value_rs DESC;
```



user_id	name	email	total_orders	lifetime_order_value_rs
int	varchar	varchar	bigint	decimal
> 24	Noor Khan	noorkhan24@mail.com	7	80970.00
> 21	Ahmed Khan	ahmedkhan21@mail.com	7	79440.00
> 180	Ali Malik	alimalik180@mail.com	7	79290.00
> 36	Fatima Iqbal	fatimaiqbal36@mail.com	7	47230.00
> 142	Ali Farooq	alifarooq142@mail.com	6	92950.00
> 113	Fatima Farooq	fatimafarooq113@mail.com	6	92760.00
> 130	Hassan Iqbal	hassaniqbal130@mail.com	6	72990.00
> 194	Ayesha Ahmed	ayeshaahmed194@mail.com	6	64990.00
> 153	Hamza Farooq	hamzafarooq153@mail.com	6	57310.00
> 1	Muhammad Abdullah	muhammadabdullah1@mail.com	6	14010.00
> 175	Fatima Ahmed	fatimaahmed175@mail.com	5	80770.00
> 30	Sara Ahmed	saraahmed30@mail.com	5	73390.00
> 53	Ahmed Iqbal	ahmediqbal53@mail.com	5	71670.00

Figure 11: Query 5 — results